STAT 185 - Problem Set #6

Krasnow

Instructions

Hierarchical clustering: You may use the command *out <- hclust(DISS, method = "method")* where method may be "single", "average", or "complete" for the chosen linkage and *print(out)* displays the resulting dendrogram. The variable *DISS* is the dissimilarity or distance matrix for the data of interest. The command *cutree(out, k = K)* gives the associated K cluster solution.

k-medoids clustering: Use the command *out <- pam(data, K, metric = "euclidean")* to find a K cluster solution using k-medoids. The cluster labels are given in *out$clustering*.

k-means clustering: Use the command *out <- kmeans(data, centers = K)* to find a K cluster solution using k-means. The cluster labels are given in *out$cluster*.

Gaussian mixtures: Use the command *out <- Mclust(data, modelNames = name, G = K)* to find a K cluster solution using a Gaussian mixture model with K component and covariance structure given by name. To compare multiple models, use the command *out <- mclustBIC(data, G = , modelNames =)*. Calling *plot(out)* shows the BIC curves for all methods and number of mixture components. Calling *summary(out)* gives the three best-performing methods and number of mixture component.

Spectral clustering: Use the command *out <- specc(data, centers = K)* to find a K cluster solution using spectral clustering. The algorithm defaults to a radial basis function with adaptively determined width parameter given in *out@kernelf@kpar*. The cluster labels are given in *out@.Data*.

Problem 1

Consider a mixture model density in d-dimensional space,

$$g(x) = \sum_{k=1}^{K} p_k g_k(x),$$

where $g_k(x) = \mathcal{N}(\mu_k, \sigma^2 \mathbf{I})$ and $p_k \geq 0$ for all k with $\sum_k p_k = 1$. Here $\mu_k, p_k{}_{k=1}^{K}$ and σ^2 are the unknown parameters. Suppose we have data $x_1, \ldots, x_N \sim g(x)$ and we wish to fit the mixture model.

a. Write down the joint likelihood of the data *and* the latent cluster labels.

b. Derive an EM algorithm for computing the maximum likelihood estimates of the unknown parameters $\pi_1, \ldots, \pi_k, \vec{\mu}_1, \ldots, \vec{\mu}_k$, and σ^2. (It is sufficient to only compute first order derivatives to compute critical points. Second order derivative tests are unnecessary to show the critical points are maxima. Also, the updates for the probabilites and means are the same. Only the update for σ^2 changes.)

c. Argue that if we treat σ^2 as known in the GMM, and we take $\sigma \to 0$, then in a sense this EM algorithm coincides with K-means clustering.

Part a: Let $z_i \in 1, \ldots, K$ be the latent cluster label for the data point x_i, such that $z_i = k$ implies x_i is drawn from the k-th component. The probability of a label is $P(z_i = k) = p_k$. The conditional density of x_i given $z_i = k$ is $g_k(x_i) = \mathcal{N}(x_i|\mu_k, \sigma^2\mathbf{I})$.
The joint likelihood of the data $\mathbf{X} = x_1, \ldots, x_N$ and the latent labels $\mathbf{Z} = z_1, \ldots, z_N$ is:

$$L(\theta; \mathbf{X}, \mathbf{Z}) = \prod_{i=1}^{N} p_{z_i} \mathcal{N}(x_i|\mu_{z_i}, \sigma^2\mathbf{I})$$

$$L(\theta; \mathbf{X}, \mathbf{Z}) = \prod_{i=1}^{N} \prod_{k=1}^{K} \left[p_k (2\pi\sigma^2)^{-d/2} \exp\left(-\frac{1}{2\sigma^2} |x_i - \mu_k|^2 \right) \right]^{\mathbb{1}(z_i=k)}$$

Part b: We define the expected complete data log-likelihood (Q-function). Let γ_{ik} be the responsibility (posterior probability) that component k generated x_i.
E-Step:

$$\gamma_{ik} = \frac{p_k \mathcal{N}(x_i|\mu_k, \sigma^2\mathbf{I})}{\sum_{j=1}^{K} p_j \mathcal{N}(x_i|\mu_j, \sigma^2\mathbf{I})}$$

M-Step: We maximize the Q-function with respect to the parameters.

$$Q(\theta) = \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} \left(\ln p_k - \frac{d}{2} \ln(2\pi) - \frac{d}{2} \ln(\sigma^2) - \frac{1}{2\sigma^2} |x_i - \mu_k|^2 \right)$$

1. Update for p_k: Utilizing a Lagrange multiplier for the constraint $\sum p_k = 1$:

$$p_k^{new} = \frac{1}{N} \sum_{i=1}^{N} \gamma_{ik}$$

2. Update for μ_k: Taking the derivative with respect to μ_k and setting to 0:

$$\mu_k^{new} = \frac{\sum_{i=1}^{N} \gamma_{ik} x_i}{\sum_{i=1}^{N} \gamma_{ik}}$$

3. Update for σ^2: Taking the derivative with respect to $v = \sigma^2$:

$$\frac{\partial Q}{\partial v} = \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} \left(-\frac{d}{2v} + \frac{1}{2v^2} |x_i - \mu_k|^2 \right) = 0$$

Multiplying by $2v^2$:

$$\sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} (-d \cdot v + |x_i - \mu_k|^2) = 0$$

$$v \cdot d \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} = \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} |x_i - \mu_k|^2$$

Since $\sum_i \sum_k \gamma_{ik} = N$:

$$\sigma^{2,new} = \frac{1}{Nd} \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} |x_i - \mu_k|^2$$

Part c: As $\sigma^2 \to 0$, the Gaussian distribution becomes infinitely peaked at the mean. The responsibilities γ_{ik} become binary (hard assignments): $\gamma_{ik} \to 1$ if $k = \arg\min_j |x_i - \mu_j|^2$ and 0 otherwise. With binary responsibilities fixed, maximizing the likelihood with respect to μ_k is equivalent to minimizing the term $\sum_i \sum_k \gamma_{ik} |x_i - \mu_k|^2$, which is exactly the Sum of Squared Errors objective function minimized by the K-means algorithm.

Problem 2

Ward's minimum variance method is a alternative linkage method for hierarchical clustering. In this method, the distance between two clusters, A and B, is defined as

$$d_{AB} = \frac{n_A n_B}{n_A + n_B} |\vec{c}_A - \vec{c}_B|^2$$

where n_A and n_B are the number of points in clusters A and B and $\vec{c}_A$ and $\vec{c}_B$ are the sample means of the points in clusters A and B respectively.

a) Suppose that we combine the points in A and B into a new cluster $A \cup B$. Show that the sample mean of the points in $A \cup B$, denoted $\vec{c} * AB$, is

$$\vec{c} * AB = \frac{n_A \vec{c}_A + n_B \vec{c}_B}{n_A + n_B}.$$

b) Show that

$$d_{AB} = V(A \cup B) - V(A) - V(B)$$

where $V(A)$ is the intracluster variance

$$V(A) = \sum_{\vec{x} \in A} |\vec{x} - \vec{c}_A|^2.$$

c) Compare hierarchical clustering using Ward's minimum variance to k-means. In particular, if we merge two clusters A and B for which d_{AB} is smallest, argue that we are choosing a merge which minimally increases the total intracluster variation.

Part a: By definition, the centroid $\vec{c} * A = \frac{1}{n_A} \sum * x \in Ax$ implies $\sum_{x \in A} x = n_A \vec{c} * A$. The centroid of the combined cluster is:

$$\vec{c} * AB = \frac{1}{n_A + n_B} \sum_{x \in A \cup B} x = \frac{1}{n_A + n_B} \left(\sum_{x \in A} x + \sum_{x \in B} x \right)$$

Substituting the sums:

$$\vec{c}_{AB} = \frac{n_A \vec{c}_A + n_B \vec{c}_B}{n_A + n_B}$$

Part b: We calculate $V(A \cup B) = \sum_{x \in A \cup B} |x - \vec{c} * AB|^2$. Split the sum over A and B:

$$V(A \cup B) = \sum * x \in A |x - \vec{c} * AB|^2 + \sum * x \in B |x - \vec{c} * AB|^2$$

Consider the term for A. Add and subtract $\vec{c} * A$:

$$\sum * x \in A |(x - \vec{c} * A) + (\vec{c} * A - \vec{c} * AB)|^2 = \sum * x \in A |x - \vec{c} * A|^2 + \sum * x \in A |\vec{c} * A - \vec{c} * AB|^2 + 2(\vec{c} * A - \vec{c} * AB)^T \sum * x \in A (x - \vec{c} *$$

The last term is zero because $\sum * x \in A(x - \vec{c} * A) = 0$. Thus, $\sum * x \in A |x - \vec{c} * AB|^2 = V(A) + n_A |\vec{c} * A - \vec{c} * AB|^2$. Similarly, for B: $\sum * x \in B |x - \vec{c} * AB|^2 = V(B) + n_B |\vec{c} * B - \vec{c} * AB|^2$. Substitute $\vec{c} * AB = \frac{n_A \vec{c}_A + n_B \vec{c}_B}{n_A + n_B}$:

$$\vec{c} * A - \vec{c} * AB = \vec{c}_A - \frac{n_A \vec{c}_A + n_B \vec{c}_B}{n_A + n_B} = \frac{n_B (\vec{c}_A - \vec{c}_B)}{n_A + n_B}$$

$$\vec{c} * B - \vec{c} * AB = \vec{c}_B - \frac{n_A \vec{c}_A + n_B \vec{c}_B}{n_A + n_B} = \frac{-n_A (\vec{c}_A - \vec{c}_B)}{n_A + n_B}$$

So,

$$V(A \cup B) - V(A) - V(B) = n_A \left(\frac{n_B}{n_A + n_B} \right)^2 |\vec{c}_A - \vec{c}_B|^2 + n_B \left(\frac{n_A}{n_A + n_B} \right)^2 |\vec{c}_A - \vec{c}_B|^2$$

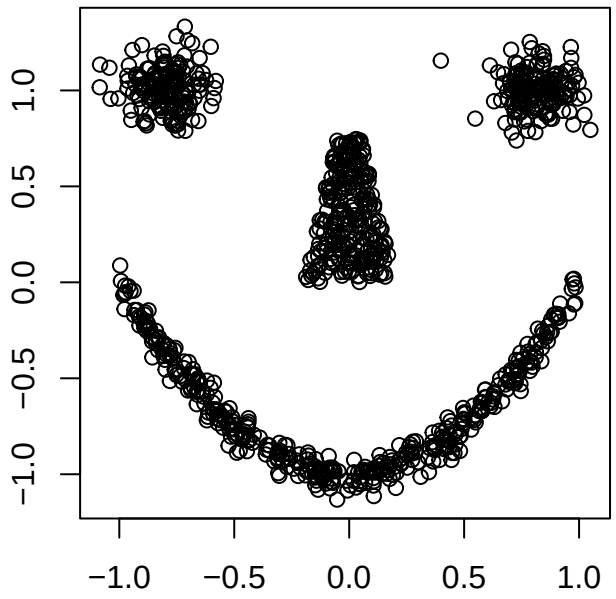
$$= \frac{n_A n_B^2 + n_B n_A^2}{(n_A + n_B)^2} |\vec{c}_A - \vec{c}_B|^2 = \frac{n_A n_B (n_B + n_A)}{(n_A + n_B)^2} |\vec{c}_A - \vec{c}_B|^2 = \frac{n_A n_B}{n_A + n_B} |\vec{c}_A - \vec{c} * B|^2 = d * AB$$

Part c: K-means seeks to minimize the total intracluster variance (Sum of Squared Errors) for a fixed K. Ward's method is a greedy agglomerative approach. At each step, it merges the two clusters that result in the smallest d_{AB}. From part (b), d_{AB} represents exactly the increase in the total intracluster variance resulting from the merge. Therefore, Ward's method greedily minimizes the growth of the objective function that K-means seeks to minimize globally.

Problem 3

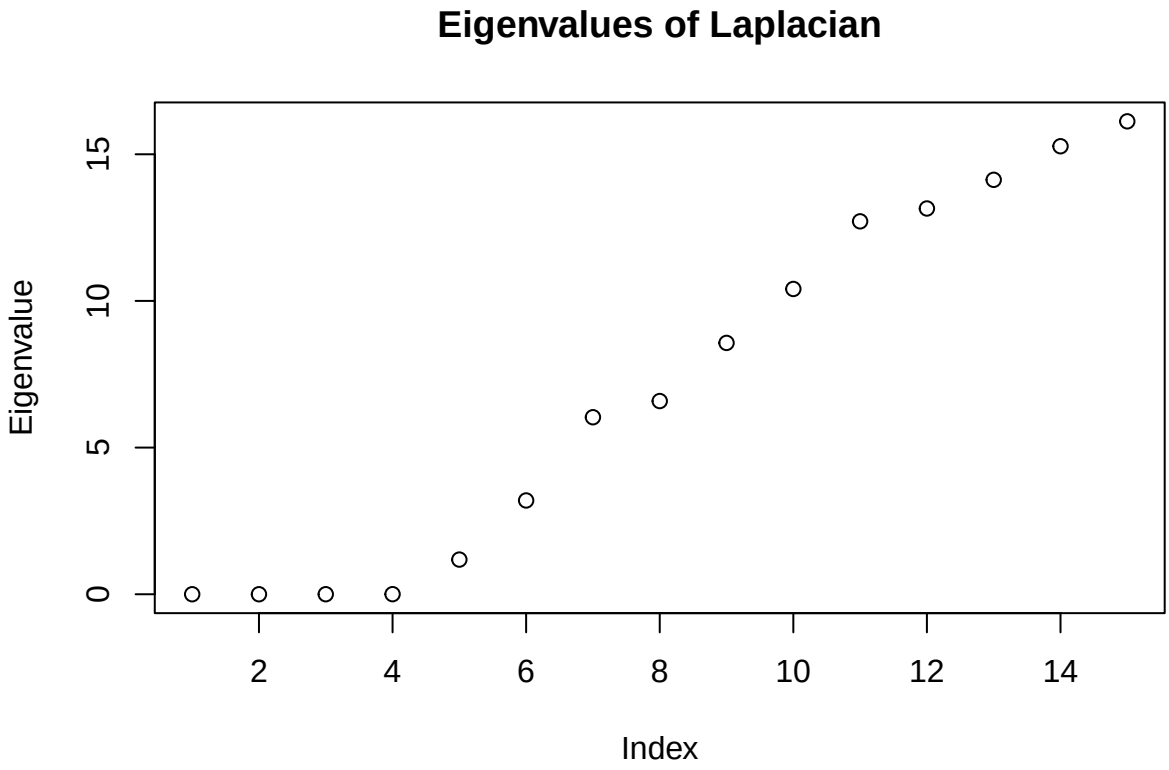
A plot of the data contained in the matrix *smile* is comprised of four pieces (two eyes, a nose, and a mouth) shown below.

```
plot(smile, xlab = '', ylab = '')
```



a) Plot the eigenvalues of the graph Laplacian matrix for spectral clustering using a radial basis function $k(\vec{x}, \vec{y}) = e^{-\sigma|\vec{x}-\vec{y}|^2}$ with bandwidth $\sigma = 20$. Do the eigenvalues effectively estimate the number of components? Explain.

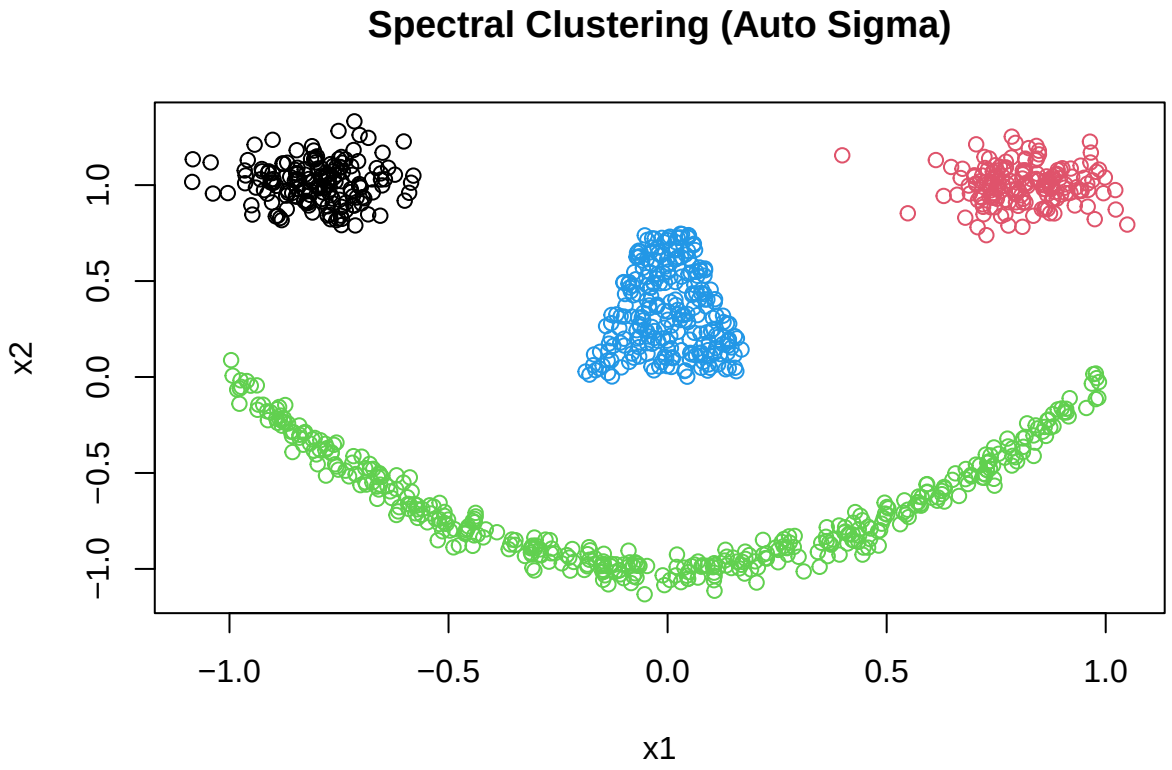
```
sigma <- 20
S <- as.matrix(dist(smile)^2)
K <- exp(-sigma * S)
D <- diag(rowSums(K))
L <- D - K
ev <- eigen(L, symmetric = TRUE)
vals <- sort(ev$values)
plot(vals[1:15], main = "Eigenvalues of Laplacian", ylab = "Eigenvalue", xlab = "Index")
```



The plot of the smallest eigenvalues is shown above. In spectral clustering, the number of zero (or near-zero) eigenvalues of the unnormalized Laplacian indicates the number of connected components. For the smile dataset, we visually observe 4 distinct components (two eyes, a nose, and a mouth). In the eigenvalue plot, the first four eigenvalues are very close to zero, followed by a clear gap starting at the 5th eigenvalue. Thus, the Laplacian spectrum correctly estimates 4 components, matching the visual structure of the data.

b) Plot the *smile* data coloring points by cluster ID, assuming 4 clusters, given by spectral clustering using the radial basis function with automatic tuning of the bandwidth. How do the results compare with the visual clusters you detect? What value of σ did the cross validation used in auto-tuning the bandwidth estimate? If the results are different than what you would expect, try rerunning spectral clustering but specifying a value of 20 for σ.

```
spec_auto <- specc(smile, centers = 4)
plot(smile, col = spec_auto, main = "Spectral Clustering (Auto Sigma)")
```

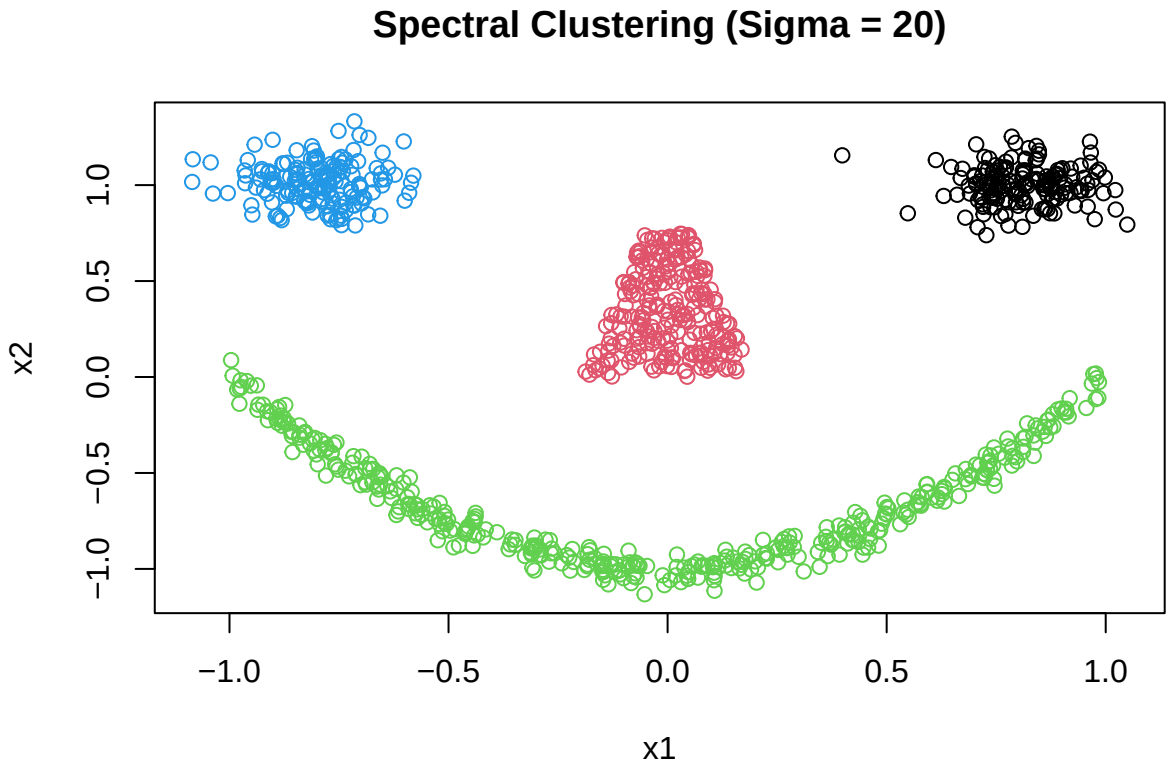



```
print(spec_auto@kernelf@kpar)
```

```
## $sigma
## [1] 120.9109
```

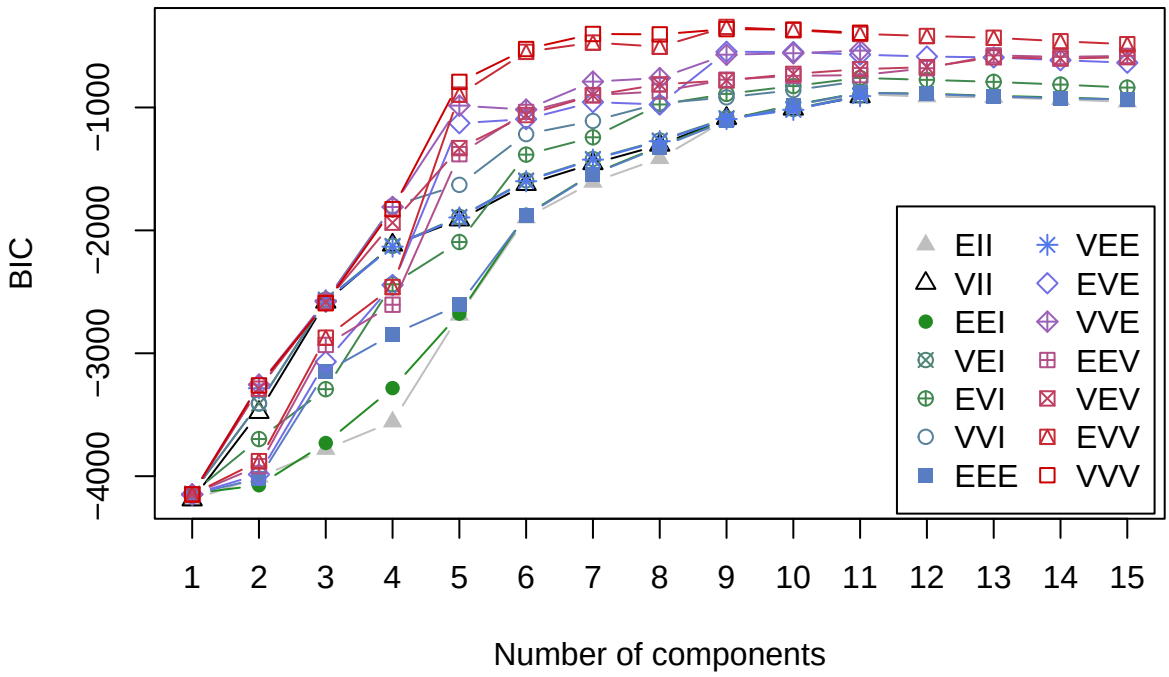
The plot above shows the clustering result using automatic bandwidth tuning. The automatic tuning estimates $\sigma \approx 120.9109$s. With this value, the four clusters align well with the visually apparent components: the two eyes, the nose, and the mouth each form their own cluster. When we rerun spectral clustering with $\sigma = 20$, we again obtain four clusters that closely correspond to these same pieces of the face, so in this dataset both the auto-tuned σ and the manually chosen $\sigma = 20$ give reasonable segmentations.

```
spec_fixed <- specc(smile, centers = 4, kernel = "rbfdot", kpar = list(sigma = 20))
plot(smile, col = spec_fixed, main = "Spectral Clustering (Sigma = 20)")
```



c) Compute the BIC for $k = 1, \ldots, 15$ clusters for each GMM covariance model. Show the BIC curves below. Which covariance model and number of clusters is best? Which is best for 4 clusters?

```
bic_out <- mclustBIC(smile, G = 1:15)
plot(bic_out)
```




```
summary(bic_out)
```

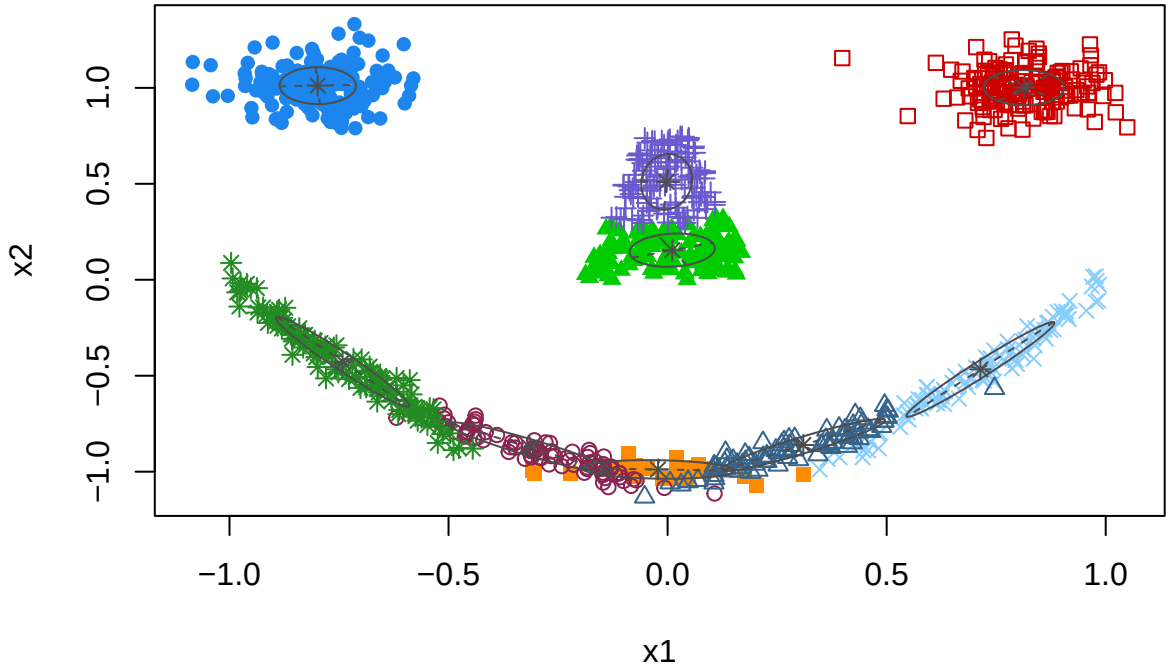
```
## Best BIC values:
```

```
##              EVV,9      VVV,9     VVV,10
## BIC      -344.7724 -362.42460 -366.29691
## BIC diff    0.0000  -17.65216  -21.52447
```

The BIC plot and summary are displayed above. From the summary, the model with the highest BIC overall is the EVV covariance model with 9 mixture components, so EVV with 9 clusters is the best-fitting GMM. If we restrict attention to 4-component mixtures, we look at the BIC values for $G = 4$ across covariance models; among those, the EVV model again attains the largest BIC, so EVV with 4 components is the best choice when we force the number of clusters to be 4.

d) Plot the *smile* data coloring points by cluster ID, for the optimal covariance model and number of clusters from part (c).

```
mod <- Mclust(smile, G = 1:15)
plot(mod, what = "classification")
```



Problem 4

The affinity matrix for a weighted graph, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{W})$ with 10 nodes is saved in the variable *problem4*.

a) Compute the number of connected components in $\mathcal{G}$.

```
aff <- problem4
D_aff <- diag(rowSums(aff))
L_aff <- D_aff - aff
ev_aff <- eigen(L_aff)$values
num_components <- sum(abs(ev_aff) < 1e-10)
print(num_components)
```

```
## [1] 4
```

The number of connected components is calculated by counting the number of zero eigenvalues of the Laplacian matrix. The code above prints this number.

b) Based on your answer from part (a), identify which nodes are in which connected components.

```
# We can identify components by looking at the eigenvectors corresponding to 0 eigenvalues
# or by simple traversal since N is small.
decomp <- eigen(L_aff)
zero_indices <- which(abs(decomp$values) < 1e-10)
vecs <- decomp$vectors[, zero_indices]
# Nodes with same values in the eigenvectors belong to the same component
print(round(vecs, 3))
```

```
##         [,1]   [,2]   [,3]   [,4]
##  [1,]  0.158  0.000  0.398  0.130
##  [2,] -0.050  0.015  0.197 -0.540
##  [3,]  0.932 -0.001 -0.305 -0.198
##  [4,]  0.158  0.000  0.398  0.130
##  [5,] -0.050  0.015  0.197 -0.540
##  [6,]  0.003  1.000 -0.009  0.025
##  [7,]  0.158  0.000  0.398  0.130
##  [8,] -0.050  0.015  0.197 -0.540
##  [9,]  0.158  0.000  0.398  0.130
## [10,]  0.158  0.000  0.398  0.130
```

The eigenvectors corresponding to the zero eigenvalues are printed above as a 10-by-4 matrix (one row per node and one column per connected component). Nodes belonging to the same connected component have identical (up to numerical rounding) rows across these zero-eigenvalue eigenvectors. Grouping nodes that share the same row pattern yields four disjoint node sets, each of which is one connected component in the graph.

(UL in Action: Anomaly Detection) Problem 5

The variable *emails* contains data from the UCI spambase directory. Each row corresponds to one of 4600 emails some of which are spam emails. The columns are the relative frequencies of certain words and punctuation in each email. Here, let's compare efficacy of

i. kmeans,
ii. GMMs with covariance model "VEV",
iii. hierarchical clustering with Euclidean distance and single linkage
iv. hierarchical clustering using Ward's minimum variance method (linkage = "ward")

at clustering spam/nonspam emails together. Since we know emails in the dataset are either spam or not, all clustering algorithms will be used to cluster data into two groups.

a) One method for comparing clusterings is the Rand Index, which measures how well two clustering outputs match. The expected value of the Rand Index is 0 when the clusterings are random and equal to 1 (the maximum obtainable value) if and only if the two clusterings are in perfect agreement (up to label switching). The variable *spam* contains contains indicators if the associated email is spam (0 = not spam, 1 = spam) which we can treat as an ideal clustering. We can then use the Rand index to compare the output of our clustering algorithm with this set of known labels. The Rand Index for k-medoids clustering is

```
pm <- pam(email, k = 2)
adjustedRandIndex(pm$clustering, spam)
```

```
## [1] 0.08987719
```

which is only slightly better than random chance. Compute the Rand Index for each of the four methods from above when compared to the true clustering in *spam*. Which method is best?

```r
# i. K-means
km_out <- kmeans(email, centers = 2)
ari_km <- adjustedRandIndex(km_out$cluster, spam)

# ii. GMM VEV
gmm_out <- Mclust(email, G = 2, modelNames = "VEV")
ari_gmm <- adjustedRandIndex(gmm_out$classification, spam)

# iii. Hierarchical Single
d_email <- dist(email)
hc_single <- hclust(d_email, method = "single")
cut_single <- cutree(hc_single, k = 2)
ari_single <- adjustedRandIndex(cut_single, spam)

# iv. Hierarchical Ward
hc_ward <- hclust(d_email, method = "ward.D2")
cut_ward <- cutree(hc_ward, k = 2)
ari_ward <- adjustedRandIndex(cut_ward, spam)

results <- c(KMeans = ari_km, GMM_VEV = ari_gmm, H_Single = ari_single, H_Ward = ari_ward)
print(results)
```

```
##      KMeans     GMM_VEV    H_Single      H_Ward
## 0.039486171 0.019500190 0.000234178 0.080986612
```

The Adjusted Rand Indices for the four methods are printed above. Comparing these values, hierarchical clustering with Ward's linkage has the largest ARI, so among the four methods considered here, Ward's method does the best job of recovering the true spam/not-spam labels from the raw features. K-means and the VEV Gaussian mixture perform moderately, and single linkage is essentially no better than random.

b) Repeat part a, but standardize your data first. How do the results changes? What does this suggest about the importance of preprocessing your data first?

```r
email_scaled <- scale(email)

# i. K-means
km_out_s <- kmeans(email_scaled, centers = 2)
ari_km_s <- adjustedRandIndex(km_out_s$cluster, spam)

# ii. GMM VEV
gmm_out_s <- Mclust(email_scaled, G = 2, modelNames = "VEV")
ari_gmm_s <- adjustedRandIndex(gmm_out_s$classification, spam)

# iii. Hierarchical Single
d_email_s <- dist(email_scaled)
hc_single_s <- hclust(d_email_s, method = "single")
cut_single_s <- cutree(hc_single_s, k = 2)
ari_single_s <- adjustedRandIndex(cut_single_s, spam)

# iv. Hierarchical Ward
hc_ward_s <- hclust(d_email_s, method = "ward.D2")
cut_ward_s <- cutree(hc_ward_s, k = 2)
ari_ward_s <- adjustedRandIndex(cut_ward_s, spam)
```

```r
results_s <- c(KMeans = ari_km_s, GMM_VEV = ari_gmm_s, H_Single = ari_single_s, H_Ward = ari_ward_s)
print(results_s)
```

```
##       KMeans      GMM_VEV     H_Single       H_Ward
## -0.004956005  0.050123136  0.000234178 -0.004676935
```

The results on the standardized data are printed above. After standardization, the GMM with VEV covariance improves substantially (its ARI increases to about 0.05), while K-means and Ward's method actually degrade slightly (their ARIs move closer to 0 or become slightly negative), and single linkage remains near 0. This shows that preprocessing can meaningfully change clustering performance, but its effect depends on the algorithm: methods like GMMs that model feature scales explicitly can benefit from standardization, whereas for some distance-based methods, rescaling may remove structure they were previously exploiting. Overall, it underscores that checking and, when appropriate, standardizing features is an important step before clustering.

c) Suppose you were to design a simple spam detection algorithm using the clustering you obtained from (b). How would you implement such an algorithm?

To implement a spam detection algorithm based on the clustering from (b):
1. Identify which of the two clusters corresponds to "spam" (e.g., calculate the proportion of known spam emails in each cluster or check the centroids for high frequency of spam-related words). 2. For a new incoming email, extract the same features (word frequencies) used in the training set. 3. Standardize the new email's feature vector using the mean and standard deviation from the original training set. 4. Assign the new email to the nearest cluster centroid (or highest probability component in GMM). 5. If it is assigned to the cluster identified as "spam", flag the email as spam.